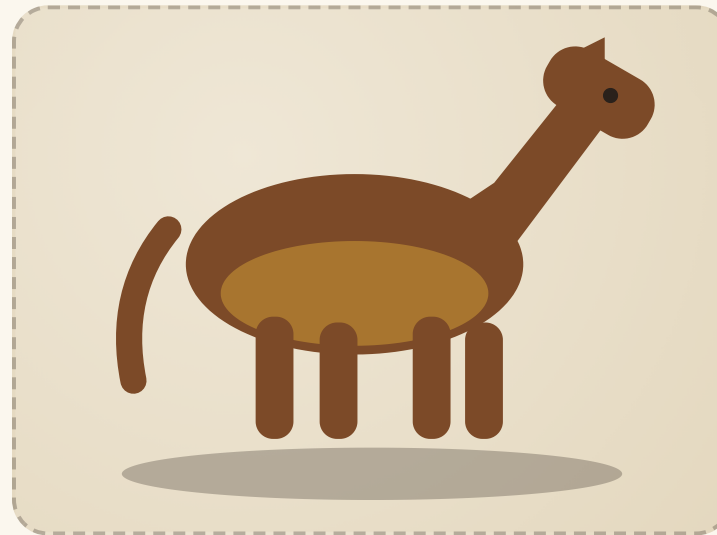


말에게 마구를 채우듯, AI에게도 마구가 필요하다



한 줄 정의

AI가 **덜 추측하고, 더 안전하게, 더 반복 가능하게** 일하도록 감싸는 작업 환경.

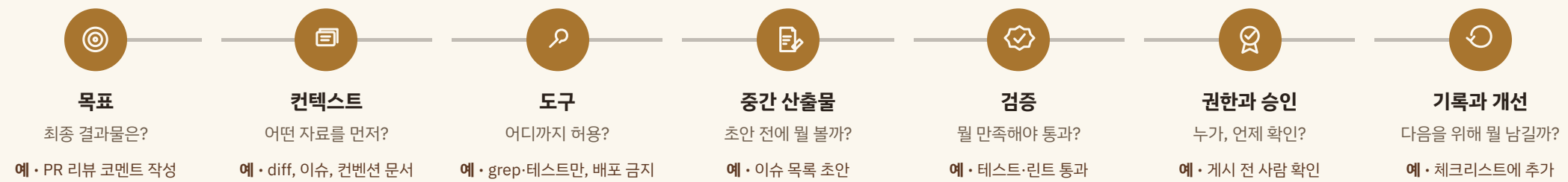
비교

프롬프트 엔지니어링 vs 하네스 엔지니어링

구분	프롬프트	하네스
범위	채팅 한 턴, 일회성	에이전트의 전체 작업 흐름
저장	세션 끝나면 사라짐	Git에 커밋되는 파일
재현성	같은 입력도 결과가 흔들림	일관된 결과를 목표로 함
공유	개인이 각자 사용	팀 전체가 공유·축적

하네스 7요소

예시: PR 코드 리뷰 자동화



왜 쓰는가

모델이 아니라 시스템을 개선한다



재현성

사람·모델이 바뀌어도 흐름은 유지

예 · 담당자가 바뀌어도 같은 체크리스트로 배포



안전성

되돌리기 어려운 행동 앞에 승인 지점

예 · 결제·삭제 전엔 반드시 사람 확인



속도

반복 업무를 매번 다시 설명하지 않음

예 · 비슷한 문의에 매번 배경 재설명 안 함



품질

"존재 확인"이 아니라 "실제로 맞물리는지"

예 · 알림 기능 추가 시, 알림 하나만이 아니라 기존 결제·로그인 흐름과 같이 써도 안 깨지는지 확인

하네스 실제 예시

```

  └─ CLAUDE.md  전역 규칙 + 자연어 라우팅 - "커밋 메시지는 한글로" 등

  └─ .claude/

    └─ rules/*.md  도메인별 세부 규칙 - nextjs.md, git-safety.md 등

    └─ agents/*.md  역할별 에이전트 카드 - code-reviewer.md, qa-engineer.md 등

    └─ skills/*/SKILL.md  오케스트레이터 · 반복 작업 매뉴얼 - git-commit 등

    └─ hooks/*.sh  세션·명령 상태 감시 훅 - git-safety-guard.sh

    └─ worktrees/{name}/  병렬 작업 격리 디렉터리 - task-t05-frontend 등

  └─ artifacts/{team}/  단계별 산출물 - 02-plan.md, 04-review.md 등

  └─ .husky/

    └─ pre-commit  커밋 전 자동 검사(훅) - 이모지 금지 등 강제
```

역할 분담 (1/2)

Task Team

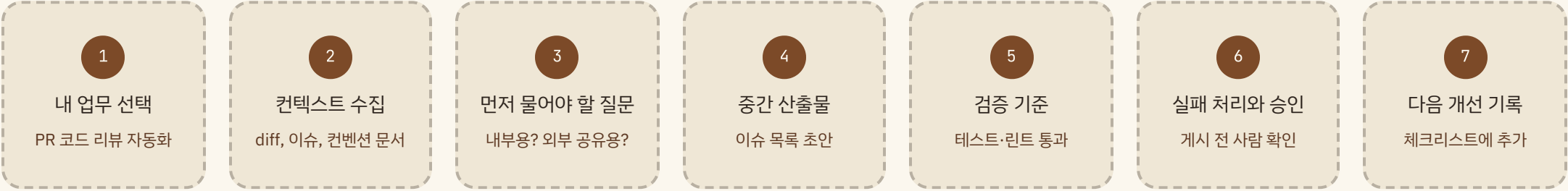


역할 분담 (2/2)

Bug Team



내 반복 업무를 하네스 카드로 바꾸기



▼ 프롬프트

다음 업무를 자동화하는 하네스를 설계해줘.

복사

- 목표: PR 코드 리뷰 자동화 (PR 리뷰 코멘트 작성까지)
- 컨텍스트: diff, 관련 이슈, 코딩 컨벤션 문서를 먼저 참고해
- 먼저 확인할 것: 이 리뷰가 내부용인지 외부 공유용인지 나한테 물어봐
- 중간 산출물: 바로 코멘트 달지 말고, 이슈 목록 초안부터 보여주고 확인받아
- 검증 기준: 테스트·린트를 통과한 코드만 리뷰 대상으로 삼아
- 실패 처리와 승인: 코멘트 게시 전에는 반드시 나한테 확인받아
- 다음 개선 기록: 이번에 놓쳤던 패턴이 있으면 체크리스트에 추가해줘

적용 예시

각 팀의 하네스 카드 활용 예시

마케팅팀

SNS 카드뉴스 초안 자동화

- 1 목표 카드뉴스 초안 3종
- 2 컨텍스트 브랜드 톤 가이드, 최근 캠페인 데이터
- 3 도구 이미지·문구 생성만, 게시 금지
- 4 중간 산출물 문구 초안
- 5 검증 브랜드 가이드 체크리스트 통과
- 6 권한과 승인 팀장 확인 후 게시
- 7 기록과 개선 반응 좋은 문구 스타일 기록

인사총무팀

채용 공고 초안 작성

- 1 목표 JD 초안 작성
- 2 컨텍스트 팀 요구사항, 기존 공고 톤
- 3 도구 문서 초안만, 사이트 게시 금지
- 4 중간 산출물 JD 초안
- 5 검증 법적 필수 문구 포함 여부
- 6 권한과 승인 채용 담당자 검토 후 게시
- 7 기록과 개선 지원자 반응 좋은 문구 축적

신기술로봇팀

로봇 제어 펌웨어 스펙 초안

- 1 목표 펌웨어 스펙 초안
- 2 컨텍스트 HW 인터페이스 문서, 기존 코드베이스
- 3 도구 정적분석만, 실기 배포 금지
- 4 중간 산출물 인터페이스 정의서 초안
- 5 검증 안전 규격·단위 테스트 통과
- 6 권한과 승인 팀장 확인 후 배포
- 7 기록과 개선 실패 사례를 회귀 테스트로 추가

주의점

하네스도 계속 다시 다듬어야 한다



간단한 일은 전체를 다 안 돌려도 된다

규모가 작은 요청까지 팀 전체 사이클을 태우면 오히려 느려진다. 직접 처리하거나 에이전트 하나만 불러서 끝낸다.

예 • 오탈자 수정, 한 줄짜리 버그 → 직접 혹은 단일 agent로 처리



초안을 그대로 계속 쓰지 않는다

같은 실수가 반복되면 그건 사람이 매번 기억해서 막을 일이 아니라 하네스 자체를 고치라는 신호다. 명세(규칙 문서)를 업데이트하거나, 기계적으로 막을 수 있으면 혹은으로 강제한다.

예 • 같은 규칙 위반이 반복 → rules/*.md에 규칙 추가 + 관련 agent 명세 업데이트 + pre-commit/혹으로 자동 검사



agent마다 model·effort를 다르게 준다

effort가 높을수록 더 깊게 생각하고 그만큼 토큰을 더 쓴다. 모든 역할에 최상위 설정을 쓰면 비용만 커지므로, 실수 비용이 큰 역할에만 높게 준다.

예 • 복잡도·실수 비용이 큰 판단(보안 검토, 설계 결정) → 상위 모델·high effort / 정형화된 저위험 작업(요약, 포맷 변환) → 하위 모델·low effort

감사합니다

출처: 하네스 엔지니어링 백과사전